设备接入网络测试管理平台方案

1.1. 整体架构设计

本方案旨在构建一个从 Excel 数据导入、平台处理、API 传输到设备指令下发的完整自动化流程。系统采用分层架构设计，确保数据流转的可靠性和高效性。

1.1.1. 系统架构概述

系统整体架构分为四个核心层次：数据输入层、平台处理层、API 传输层和设备执行层。数据从 Excel 文件开始，经过平台解析和字段分解，通过 API 接口传递给 Python 脚本，最终由 Python 脚本控制设备或 TSMaster 软件执行指令，并将执行结果回传至平台。

1.1.2. 数据流向设计

Excel 文件 → 平台上传 → 字段解析与分解 → API 接口 → Python 脚本 → 设备/TSMaster → 结果回传。

整个流程形成一个完整的闭环，确保指令下发和结果反馈的实时性和准确性。

1.2. Excel 数据导入与平台处理

1.2.1. Excel 数据准备与导入

用户首先准备包含配置参数的 Excel 文件。为提高数据质量，建议采用模板导入方式，即提供预定义的 Excel 模板，确保字段格式、数据类型与平台要求一致。模板应包含明确的列标题，并在第二行或批注中给出校验规则和示例。平台支持常见的文件格式（xls、xlsx），并对文件大小和单 sheet 导入进行限制。

1.2.2. 字段解析与分解流程

平台接收到上传的 Excel 文件后，启动解析流程。首先进行文件解析，判断文件格式是否正确，避免大文件导致的内存溢出问题。解析完成后，进行表头校验，将 Excel 列名与平台预定义的字段进行映射比对，确保字段对应关系准确。

对于需要进行字段分解的场景（例如一个单元格内包含多个用分隔符连接的参数），平台需调用专门的拆分逻辑。这可以通过正则表达式、字符串分割函数或类似 Excel“文本分列”的功能实现，将复合信息拆分为独立的、结构化的字段，为后续 API 传输做好准备。

所有校验结果需即时反馈，对于校验失败的数据，应明确标识出错的行号、列号及具体原因，并提供错误文件下载地址。

1.2.3. 通过 API 向 Python 脚本传递数据

平台在数据校验通过后，将结构化数据通过 API 接口传递给 Python 脚本。数据以 JSON 格式进行传输，JSON 没有长度限制且能很好地表示嵌套和复杂的对象结构。对于包含多条配置记录的数据，使用请求体(Request Body)以 JSON 数组形式传递。

API 通信使用身份认证（如 API Key、OAuth Token）和授权机制，敏感信息应存储在环境变量中而非硬编码。

平台记录详细的操作日志，满足数据溯源和合规要求。

1.2.4. Python 脚本实现与设备控制

1.2.4.1. Python 脚本架构

Python 脚本作为系统的执行引擎，负责从平台获取配置数据，并根据配置类型分发到不同的目标系统。脚本核心功能包括平台 API 客户端、配置解析器、目标系统适配器、执行引擎。

1.2.4.2. 平台 API 客户端实现

Python 脚本调用平台 API 接口。需要实现完整的错误处理、重试机制和日志记录功能。关键实现包括身份验证、数据获取、异常处理等。

1.2.4.3. 设备控制实现方式

根据设备类型和通信协议，Python 脚本采用不同的控制方式：

对于支持网络通信的设备，可以使用相应的协议进行直接通讯；

对于需要通过 TSMaster 软件控制的设备，Python 脚本可以通过 TSMaster 提供的 RPC 接口或 COM 组件进行交互。

本方案通过将 Excel 导入、平台智能解析、API 标准化传输与 Python 灵活执行相结合，实现了配置参数从录入到执行的端到端自动化。它显著提升了批量参数设置的效率和准确性，降低了人工操作错误的风险；通过分层设计和标准化接口，具备了良好的灵活性和可扩展性，能够适应不同设备和控

制软件的需求；同时，完善的数据校验、安全控制和日志记录机制，确保了流程的可靠性和可追溯性。

1.3. 网络自动化测试系统机柜接入

该测试系统测试执行软件为 CANoe10 ，CANoe10 文档中没有说明可以用 python 脚本调用，需要通过开发 C#脚本调用；已拿到该项目原测试软件的 C#源码，更改程序增加与网络测试管理平台对接的 API 接口，同时增加用例与脚本匹配策略程序；

1.4. 便携式网络自动化测试系统机柜接入

该测试系统现由 TAMS 软件进行自动化控制，同时该执行软件为 CANoe17 ，基于上述基础条件，有以下两种方案；

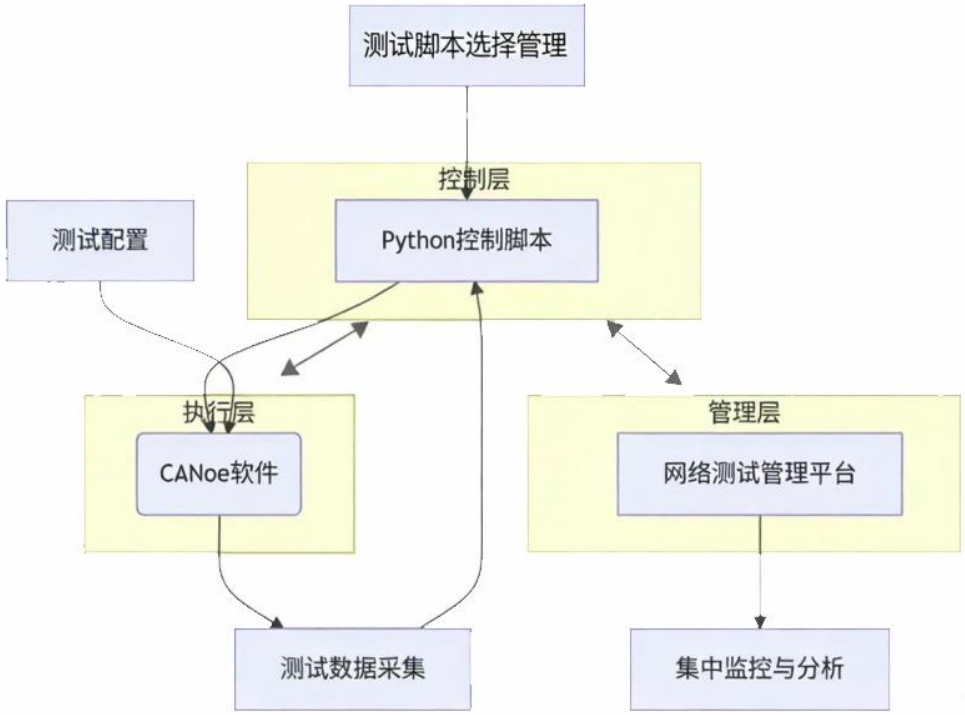
方案一：基于现有 TAMS 软件接口开发

该方案需要 TAMS 软件支持相关 API 接口的开放，才可进行二次开发（待厂家确认 TAMS 软件是否开放相关 API 接口）

方案二：基于 CANoe 的接口文档，进行二次开发

可行性背景：CANoe 提供的 COM 接口使得外部脚本能够访问或控制 CANoe 软件，从而实现自动化测试任务，而易用且具有丰富生态的 Python 无疑是一个很好的选择。接下来我们将使用 Python 访问 CANoe COM 以实现自动化测试的基本方法，通过几个常用的 COM 对象及方法，构建了简单的自动化测试示例工程。对于更复杂的工程，同样可以根据基本方法，灵活运用 COM 以满足测试需求。进一步，可以将 CANoe COM 封装成 Python 库，利用可复用性高效地完成针对不同需求的自动化测试。

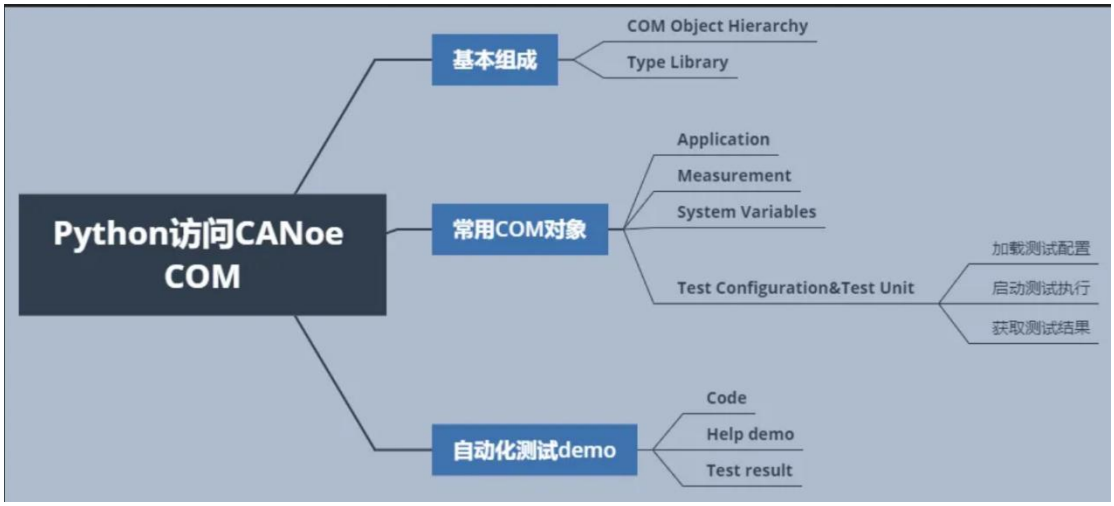
流程：测试机柜电脑安装客户端软件，该软件首先负责与测试管理平台通讯获得具体测试任务，根据任务控制执行软件选择特定脚本进行测试执行，其次通过调用 API 接口获得测试结果，然后将测试结果上传给测试管理平台。



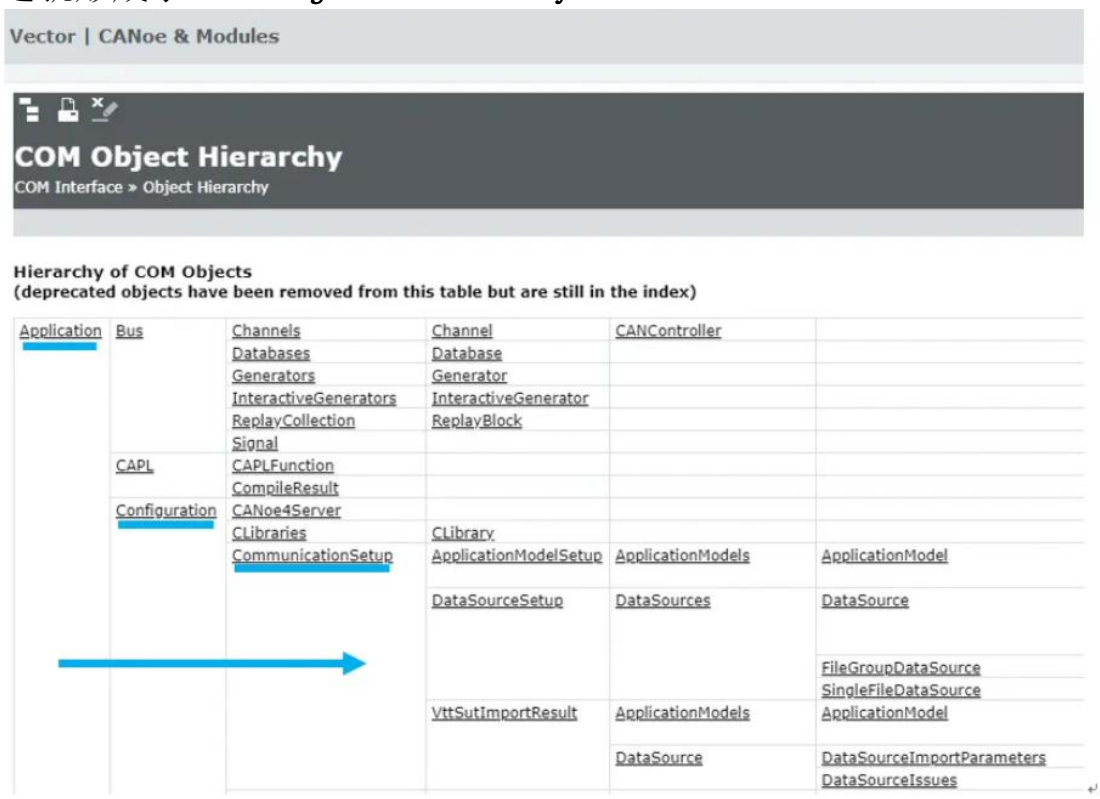
1.4.1. COM 介绍

COM 全称 Component Object Model，是微软为 Windows 平台软件提出的、实现软件之间互操作的标准。它不会规定软件的具体实现，而是声明一种对象模型，使得满足这种模型的对象之间能交互，这些对象通常被称为组件（Component）。组件会实现特定的功能，而这些功能以特定的方式提供——即接口（Interface），其他组件通过接口使用它们。另外，组件需要经过注册（Registry），才能被其他软件发现和使用。注册后的组件向其他软件提供服务，因此组件将作为服务端（COM Server），其他想要使用服务的作为客户端（COM Client）。

在安装完成 CANoe 软件后，CANoe 已经在 Windows 组件服务管理器中注册了 CANoe COM Server，如果需要重新注册，可以在安装目录下（默认为 C:\Program Files\Vector CANoe 14）的 Exec64 文件夹中找到 RegisterComponents.exe，运行即手动注册。



在 CANoe 中，各个功能模块按照一定的层级组织在一起构成整个 CANoe 软件。与这些功能模块直接相关的 COM 组件同样也是按照相应的层级组织的，这就形成了 COM Object Hierarchy：



Vector | CANoe & Modules

COM Object Hierarchy

COM Interface » Object Hierarchy

Hierarchy of COM Objects
(deprecated objects have been removed from this table but are still in the index)

Application	Bus	Channels	Channel	CANController	
		Databases	Database		
		Generators	Generator		
		InteractiveGenerators	InteractiveGenerator		
		ReplayCollection	ReplayBlock		
		Signal			
	CAPL	CAPLFunction			
		CompileResult			
	Configuration	CANoe4Server			
		CLibraries	CLibrary		
		CommunicationSetup	ApplicationModelSetup	ApplicationModels	ApplicationModel
			DataSourceSetup	DataSources	DataSource
					FileGroupDataSource
					SingleFileDataSource
			VttSutImportResult	ApplicationModels	ApplicationModel
				DataSource	DataSourceImportParameters
					DataSourceIssues

在 CANoe 软件安装目录下（默认为 C:\Program Files\Vector CANoe ）的 Exec32\COMdev 目录中包含了注册 COM 所用的类型库，如下图所示 CANoe.h 头文件：

名称	修改日期	类型	大小
CANalyzer.h	2020/10/20 3:16	C/C++ Header	3,785 KB
CANalyzer.tlb	2020/10/20 3:16	TLB 文件	343 KB
CANalyzer_i.c	2020/10/20 3:16	C Source	77 KB
CANoe.h	2020/10/20 3:16	C/C++ Header	3,785 KB
CANoe.tlb	2020/10/20 3:16	TLB 文件	343 KB
CANoe_i.c	2020/10/20 3:16	C Source	77 KB
Diag.h	2020/10/20 3:16	C/C++ Header	53 KB
Diag_i.c	2020/10/20 3:16	C Source	3 KB
Diagnostic.tlb	2020/10/20 3:16	TLB 文件	8 KB
Media.tlb	2020/4/8 17:40	TLB 文件	72 KB
Media_i.c	2020/4/8 17:40	C Source	4 KB
Media_i.h	2020/4/8 17:40	C/C++ Header	68 KB

在 CANoe.h 头文件中包含所有 CANoe COM 对象的接口定义，比如 ITestConfiguration 接口：

```
EXTERN_C const IID IID_ITestConfiguration;

#if defined(__cplusplus) && !defined(CINTERFACE)

    MIDL_INTERFACE("41AA0642-6523-473C-8A58-57123E020859")
    ITestConfiguration : public IDispatch
    {
    public:
        virtual /* [helpstring][id][propget] */ HRESULT STDMETHODCALLTYPE get_Application(
            /* [retval][out] */ IDispatch **pVal) = 0;

        virtual /* [helpstring][id][propget] */ HRESULT STDMETHODCALLTYPE get_Parent(
            /* [retval][out] */ IDispatch **pVal) = 0;

        virtual /* [helpstring][id][propget] */ HRESULT STDMETHODCALLTYPE get_Enabled(
            /* [retval][out] */ VARIANT_BOOL *bEnabled) = 0;

        virtual /* [helpstring][id][propput] */ HRESULT STDMETHODCALLTYPE put_Enabled(
            /* [in] */ VARIANT_BOOL bEnabled) = 0;

        virtual /* [helpstring][id][propget] */ HRESULT STDMETHODCALLTYPE get_Name(
            /* [retval][out] */ BSTR *pVal) = 0;

        virtual /* [helpstring][id][propput] */ HRESULT STDMETHODCALLTYPE put_Name(
            /* [in] */ BSTR newVal) = 0;

        virtual /* [helpstring][id][propget] */ HRESULT STDMETHODCALLTYPE get_Verdict(
            /* [retval][out] */ eVerdictState *pVal) = 0;

        virtual /* [helpstring][id][propget] */ HRESULT STDMETHODCALLTYPE get_TestUnits(
            /* [retval][out] */ ITestUnits **pVal) = 0;

        virtual /* [helpstring][id][propget] */ HRESULT STDMETHODCALLTYPE get_Report(
            /* [retval][out] */ ITestConfigurationReport **pVal) = 0;

        virtual /* [helpstring][id] */ HRESULT STDMETHODCALLTYPE Start( void) = 0;
```

1.4.2. python 控制 CANoe 自动化测试示例

CANoe 通过 COM(Component Object Model)接口提供自动化控制能力。这意味着你可以使用支持 COM 调用的编程语言(如 Python)像操作一个软件对象一样远程控制 CANoe 的各种功能。以下是实现这一过程的具体步骤和代码示例。

1.4.2.1. 连接与基础控制

首先确保你的环境准备就绪，并建立 Python 与 CANoe 之间的连接。

(1)环境准备与初始化：安装必要的 Python 库，主要是用于 COM 交互的 pywin32:

```
pip install pywin32
```

使用以下代码连接至 CANoe:

```python
import win32com.client
import pythoncom
import time

class CANoeController:
    def __init__(self):
        self.app = None
        self.measurement = None

    def start_canoe(self, cfg_path=None):
```

(2)测量控制:

控制测量的开始和停止是自动化测试的基本操作:

```python
def start_measurement(self):
    """开始测量"""
    if self.app and not self.measurement.Running:
        self.measurement.Start()
        # 等待测量真正启动
        for i in range(10):
            if self.measurement.Running:
                print("测量已启动")
                return True
            time.sleep(0.5)
```

注意:在开始和停止测量后,建议添加简单的等待循环,以确保状态切换完成,避免后续操作时序错误。

1.4.2.2. 数据交互操作

一旦建立了连接并启动了测量，就可以进行更深入的数据交互。

(1)信号读写:

通过 CANoe 的 COM 接口，你可以读取或设置总线上特定信号的值。需要注意的是，成功设置信号值的前提通常是该信号所属的报文是通过 CANoelL(Interaction Layer)发送的，而非通过 IG(Interactive Generator)等其他方式。

```python
def read_signal(self, channel, message_name, signal_name, bus_type="CAN"):
    """读取信号值"""
    try:
        if bus_type.upper() == "CAN":
            bus = self.app.BusSystems(channel) # 获取指定通道的总线系统
        # 对于其他总线类型（如FlexRay），可能需要使用不同的属性，例如app.FlexRay
        # bus = self.app.FlexRay  # 对于FlexRay总线

        signal_value = bus.Signals.Item(signal_name).Value # 读取信号值
        print(f"信号 {signal_name} 的值: {signal_value}")
```

(2)环境变量与系统变量操作:

环境变量和系统变量是 CANoe 中用于模块间通信或模拟外部环境的重要元素。

```python
def get_environment_variable(self, var_name):
    """获取环境变量的值"""
    try:
        env_var = self.app.EnvironmentVariables.Item(var_name)
        value = env_var.Value
        print(f"环境变量 {var_name} 的值为: {value}")
        return value
    except Exception as e:
        print(f"获取环境变量失败: {e}")
        return None
```

展开

1.4.2.3. 完整应用示例

下面是一个综合性的示例，演示如何将上述功能组合起来完成一个简单的自动化测试流程。

```python
def demo_automated_test():
    """演示一个完整的自动化测试流程"""
    canoe = CANoeController()
    cfg_path = r"C:\Your_Project_Path\your_config.cfg" # 替换为你的配置文件路径

    try:
        # 1. 启动CANoe并加载配置
        if not canoe.start_canoe(cfg_path):
            return

        # 2. 启动测量
        canoe.start_measurement()
        time.sleep(2) # 等待系统稳定

        # 3. 设置环境变量以触发某些操作 (例如激活测试节点)
        canoe.set_environment_variable("TestEnable", 1)

        # 4. 模拟信号输入 (例如设置车辆速度信号)
        canoe.write_signal(1, "VehicleData", "VehSpd", 50.0) # 设置车速为50 km/h
        time.sleep(1)

        # 5. 验证ECU响应 (例如读取刹车灯状态信号)
        brake_light_status = canoe.read_signal(1, "BodyData", "BrakeLight")
        if brake_light_status == 1:
            print("ECU响应正常: 刹车灯已亮起")
        else:
            print("ECU响应异常: 刹车灯未亮起")

        # 6. 停止测量
        canoe.stop_measurement()

    except Exception as e:
        print(f"自动化测试过程中发生错误: {e}")
    finally:
        # 确保在退出前停止测量
        if canoe.measurement and canoe.measurement.Running:
            canoe.stop_measurement()
        # 可选: 退出CANoe
        # canoe.quit_canoe()

# 运行演示
if __name__ == "__main__":
    demo_automated_test()
```

1.4.2.4. 重要注意事项与常见问题

1.COM 接口可用性:确保你的 CANoe 版本支持 COM 自动化接口。较新的版本(如 CANoe11 及以上)通常提供更好的支持。

2.权限与路径:以管理员身份运行Python脚本有时是必要的,特别是当CANoe也需要管理员权限时。确保提供的配置文件路径、日志文件路径等是存在且可访问的。
3.时序与同步:在脚本中适当使用 time.sleep()或基于事件的等待机制(例如等待某个系统变量或信号变为预期值),以确保CANoe有足够的时间响应上一个命令。
4.错误处理:如上例所示，使用 try-except 块来捕获和处理可能出现的异常(如COM调用失败、对象未找到等)，增强脚本的健壮性。
5.信号写入条件:如前面提到的，通过 write_signal 设置信号值，通常需要确保该信号所属的报文是通过CANoelL(Interaction Layer)发送的。如果报文是通过IG(Interactive Generator)等其他方式发送的，直接设置信号值可能会失败。

1.5. 车载以太网测试系统机柜接入

车载以太网测试系统机柜（包含思博伦、CANOE等设备），需要将各个分离的设备，完成集成自动化测试，这就需要开发测试软件系统分别调用各个设备，同时自动化更改配置配置信息，完成自动化测试；

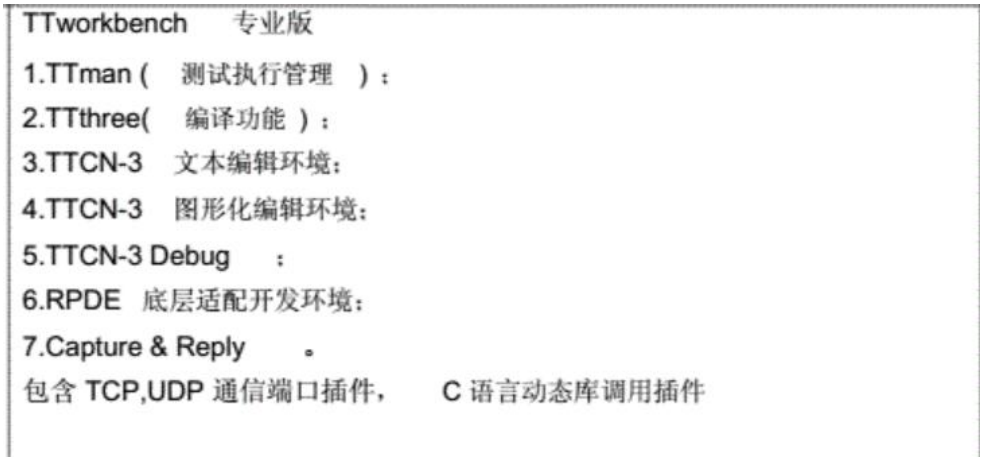
方案一：调用Ttworkbench软件接口实现

TestCenter软件主要进行设备的配置，Ttworkbench软件主要进行测试自动化，TTworkbench是调用TestCenter的Java API来控制C50的，TTworkbench的接口文档可以从软件的help界面导出。

TTworkbench软件可通过命令行的方式去调用（python脚本调用命令行API），网络测试管理平台调用TTworkbench软件接口，可进行TC8协议一致性测试，但TSN测试时需要有信息交互和参数输入确认，这个思博伦软件接口实现困难。

需要确认参数配置是否有相关接口（已确认不支持）。

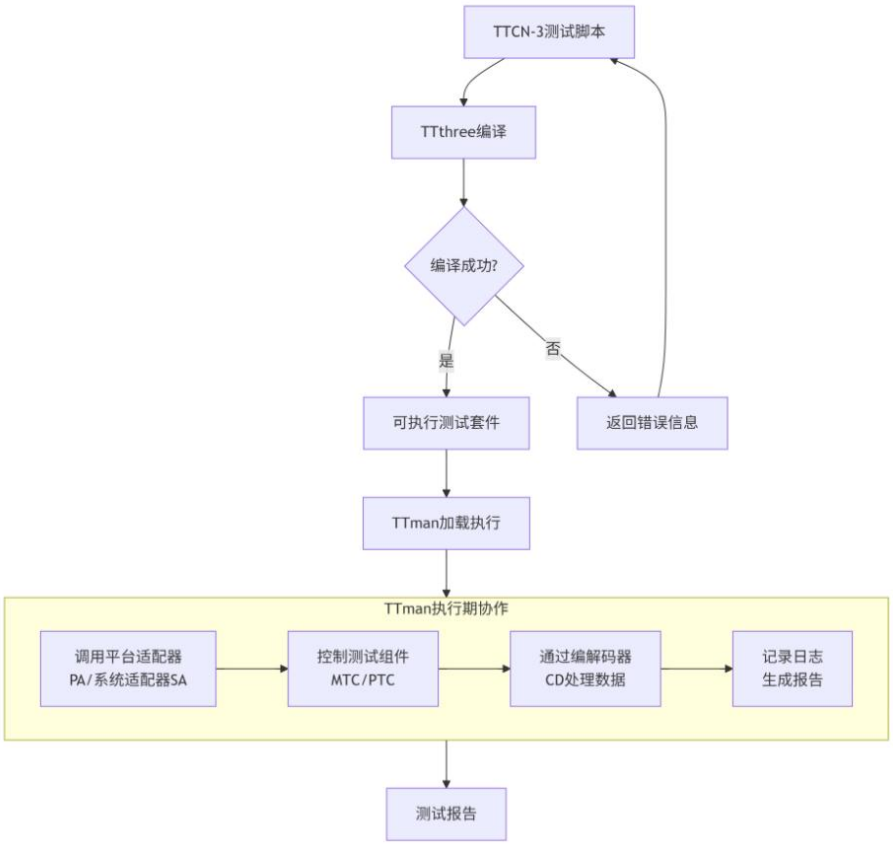
TTworkbench的命令行控制围绕 TTman和 TTthree等核心组件构建；



TTman（测试执行管理器）：这是核心命令，负责加载编译好的测试套件、调度测试用例执行、并生成测试报告。

TTthree（编译器）：在运行测试前，通常需要先用 TTthree 编译器将 TTCN-3 源代码（可能是文本或图形格式）编译成可执行代码。

系统适配器（SA）和平台适配器（PA）：这些组件负责测试系统与被测设备（DUT）之间的底层通信（如发送以太网报文）和外部交互（如定时器管理）。它们的配置通常由项目文件或环境变量定义，是测试能够正确执行的基础。



TTsuite 测试套装是调用 API 接口必须基础条件：

测试套装（TTsuite），包括 OPEN Alliance SIG 一致性测试（TC8），汽车 AVB 一致性测试，AUTOSAR 一致性测试等套装，每个测试套装都包含多种经过验证的测试用例，配合 TTworkbench，能够实现车载以太网常见协议的一致性测试的自动化执行。

调用 TTworkbench 接口示例：

包装 com.testingtech.tworkbench.ttman.server.api

TTman Server API 可远程访问 TTworkbench 的测试管理和执行功能。

参见:描述

界面摘要

接口	描述
批量执行功能	处理批量执行任务(例如
竞选雕像	
执行营销	负责竞选执行工作
ExecuteControlJob	用于控制执行
ExecuteTestCaseJob	处理用于测试用例执行
竞选执行	代表执行测试活动的工作队列。
射击游戏执行手册	活动执行事件
执行基地	用于远程访问 TTman 执行服务器和 TTman 执行队列的常见 API。
执行手册	从TTman服务器到客户端的异步反馈接口。
执行服务器	远程访问TTman执行服务器的主API。
项目经理	
IProjectPublisher	项目发布服务
星际隐秘	请求通过SUT操作或TTLBRAY的测试执行方式触发函数为 measyBox 和 timedMessageBox。
意大利语	对测试执行所提出的问题的回答。
工作	执行服务器任务的处理工具。
状态	用于响应状态ITEResponse。

测试过程：

1、环境准备

- 连接 Test System 与 DUT
- 加载对应的 TTsuite
- 配置 TTsuite 参数，如 IP 地址，MAC 地址等
- 启动 UT

2、执行测试

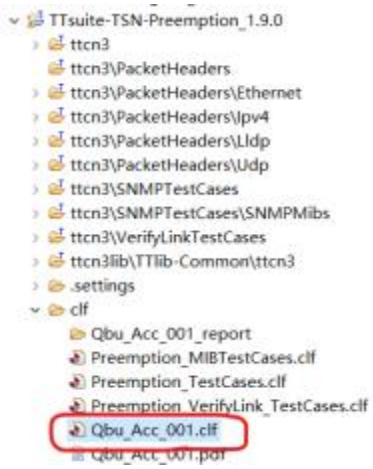
- 运行测试脚本
- 获得测试数据和测试报告

使用TTman 命令行调用TTworkbench clf 进行测试

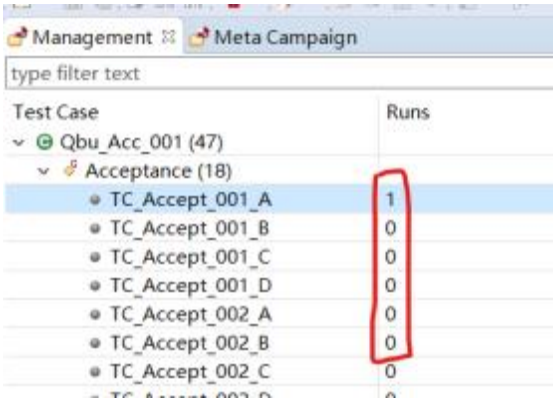
1、在 TTworkbench 客户端上创建“可执行的clf”

先通过手动方式在TTworkbench 客户端上创建可执行的clf，包括测试仪参数、配置参数、测试用例执行次数等，例如：

- 在CFL 文件夹中复制一个“.clf”文件并重命名成某个测试用例的名字

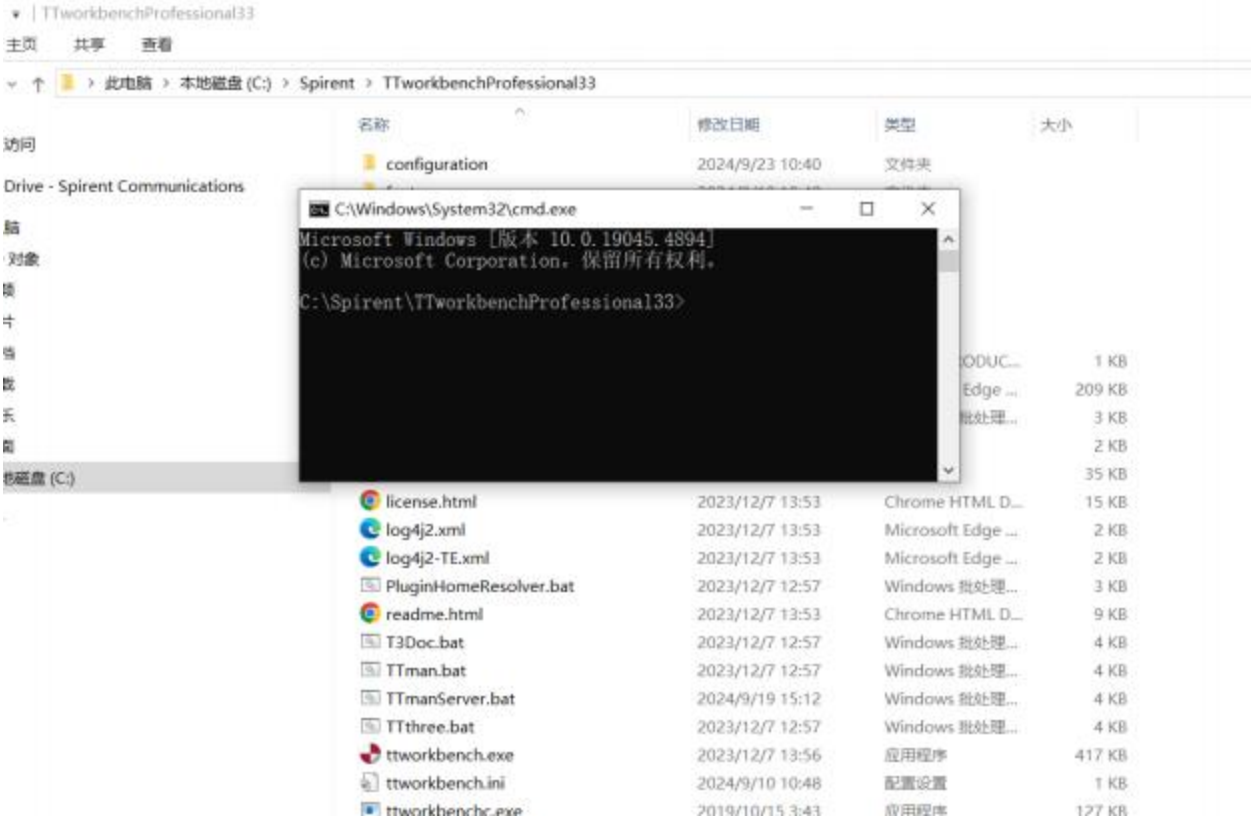


- 双击加载该clf，配置Parameters 并进行调试，确保case 可以正常执行；然后把这个clf 文件对应的case 的Runs 参数设置为 1，其他case 全部设置为0（确保使用TTman 命令调用clf 的时候只会执行这一个case）。依次类推，再创建第二个clf 文 件，再设置对应的Runs 参数，每个clf 文件对应一个可执行case。



2、关闭 TTworkbench，使用 TTman 命令调用该clf

- 在TTworkbench 安装目录下启动cmd 控制台



- 使用TTman.bat 脚本调用clf 文件进行测试，指令格式如下：

TTman.bat --data <workspace_Path> --log C:\Spirent\Logs -r pdf --report-dir C:\Spirent\Logs <CLF_Path>\DinoListTestControl.clf

举例:

TTman.bat --data C:\Users\lzheng\workspace33 --log C:\Spirent\TTworkbenchLogAndReport\Logs -r pdf --report-dir C:\Spirent\TTworkbenchLogAndReport C:\Users\lzheng\workspace33\TTsuite-TSN-Preemption_1.9.0\clf\Qbu_Acc_001.clf

上面命令中红色部分用于指定 workspace 的路径，蓝色部分用于指定把测试完毕的tlz log 导出到哪个路径，绿色部分用于指定测试report 的导出路径和格式，黄色部分为要调用的某个 clf 文件进行测试

- 执行过程在cmd 会有log 打印

```
C:\Spirent\TTworkbenchProfessional33>TTman.bat --data C:\Users\lzheng\workspace33 --log C:\Spirent\TTwo
rkbenchLogAndReport\Logs -r pdf --report-dir C:\Spirent\TTworkbenchLogAndReport C:\Users\lzheng\workspa
ce33\TTsuite-TSN-Preemption_1.9.0\clf\Qbu_Acc_001.clf
MAN-EP expires in 166 days
TTWORKBENCH_SUPPORT expires in 166 days
TT3-EP expires in 166 days
Key repository file not present, ignored.
adding libraries for plugin RTPPortPlugin
10:36:59.701: parsing given loader file 'C:\Users\lzheng\workspace33\TTsuite-TSN-Preemption_1.9.0\clf\Q
bu_Acc_001.clf'
loaded module SNMPv1v2c_Configuration
loaded module SNMPv2_PDU
loaded module SNMPv2_SMI
loaded module ASNAUX
loaded module SNMPv1v2c_Constants
loaded module Preemption_Parameters
loaded module Vlan_Types
loaded module LibCommon_BasicTypesAndValues
loaded module LibCommon_DataStrings
loaded module Preemption_Types
loaded module EthernetFrame_Types
loaded module LldpPacket_Types
```

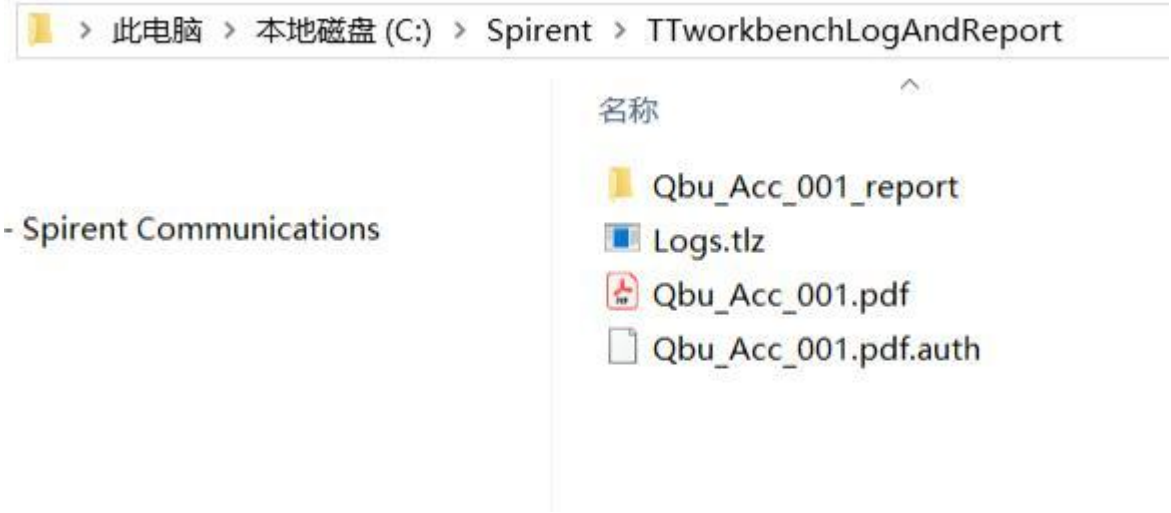
- Log 上出现Test execution finished，测试结束

```
Save configuration to [TC_Accept_001_B_STC_2024-09-23-10-38-47.xml]
Starting Stream
Current generator state RUNNING
Stop Generator
Current generator state STOPPED
Stop Stream
Retrieving Captured frames...
Retrieving Captured frames...
Captured frames:        6
Releasing ports ...
Disconnect from the chassis ...
Deleting project
10:39:22.578: TC_Accept_001_B -> pass [Reason: "All frames are received in expected order at TS2"]
STCProxy: Shutdown
STCProxy: Done

Total number of executed test cases: 2
pass:     2 = 100%


Test execution finished. Please find the created log file at: C:\Spirent\TTworkbenchLogAndReport\Logs.t
lz
WARNING: An illegal reflective access operation has occurred
WARNING: Illegal reflective access by com.lowagie.text.pdf.MappedRandomAccessFile$1 (file:/C:/Spirent/T
TworkbenchProfessional33/plugins/com.lowagie.text_2.1.7.v201004222200.jar) to method java.nio.DirectByt
eBuffer.cleaner()
WARNING: Please consider reporting this to the maintainers of com.lowagie.text.pdf.MappedRandomAccessFi
le$1
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective access operations
WARNING: All illegal access operations will be denied in a future release
Test report generation finished. Please find the created test report at: C:\Spirent\TTworkbenchLogAndRe
port\Qbu_Acc_001.pdf
```

3、在导出路径中找到tlz log 和report 文件



方案二：Python 脚本直接控制 C50（不可行）

步骤	描述
1. 安装必要的库	安装Python和必需的库
2. 连接设备	使用TCP/IP连接到思博伦测试仪
3. 发送命令	使用Python发送测试命令
4. 接收数据	接收测试仪返回的数据
5. 处理数据	解析并处理返回的数据
6. 断开连接	结束后关闭连接

详细步骤与代码实现

1.安装必要的库

首先，确保你已经安装了 Python 和 pyvisa 库，pyvisa 是一个用于控制仪器的 Python 库。可以使用以下命令安装:

```
pip install pyvisa
```

2.连接设备

使用 Python 连接到思博伦测试仪。通常我们是通过 TCP/IP 协议进行连接的。以下是示例代码:

```python
import pyvisa

# 创建资源管理器
rm = pyvisa.ResourceManager()
# 设备的IP地址和端口号，需根据具体情况调整
device_ip = "TCPIP::192.168.1.100::INSTR"
# 连接到思博伦测试仪
try:
    device = rm.open_resource(device_ip)
    print("连接成功！")
except Exception as e:
    print(f"连接失败: {e}")
```

这段代码创建了一个资源管理器，并通过指定的 IP 地址连接思博伦测试仪。注意，device_ip 需要根据你的设备进行适当修改。

3.发送命令

连接成功后，你可以向设备发送各种命令来控制其操作。以下是一个发送命令的示例

```python
# 发送初始化命令
initialize_command = "*RST"  # 重置设备
device.write(initialize_command)  # 发送命令
print("已发送初始化命令。")
```

这段代码使用*RsT 命令重置设备。你可以根据需求更改发送的命令。

4.接收数据

发送命令之后，有时你需要从设备获取状态或数据。可以使用以下代码:

```python
# 询问设备状态
status_command = "*IDN?"  # 查询设备标识
device_id = device.query(status_command)  # 发送命令并接收返回
print(f"设备标识: {device_id}")
```

使用*IDN?命令来查询设备的标识信息，并将返回的结果储存在 device_id 变量中。

5.处理数据

收到数据后，可以根据实际需要进行处理。以下是一个简单的示例:

```python
# 假设返回的数据是以逗号分隔的值
data = device.query("MEASURE:ALL?")  # 获取所有测量数据
parsed_data = data.split(',')  # 分割并解析数据
print("解析后的数据:", parsed_data)
```

这里假设设备返回的数据是以逗号分隔的，通过 split)函数进行解析。从而可以进一步处理这些数据。

6.断开连接

最后，在完成所有操作后，别忘了关闭与设备的连接:

```python
# 关闭设备连接
device.close()
print("连接已关闭。")
```

这段代码将关闭与思博伦测试仪的连接，保持良好的编程习惯。